FACULTY OF COMPUTER SCIENCE & AI

IMPLEMENTATION OF AES-128 IN CTR MODE

Table of Contents

Table of Contents 2
Table of Figures 3
Abstract 4
1. Introduction 5
1.1 Key Technical Terms 5
1.2 Aims and Objectives 5
2. Background 6
2.1 The AES Algorithm 6
2.2 Key Expansion 6
2.3 CTR Mode of Operation 6
3. Development 8
3.1 Class Architecture 8
3.2 Helper Functions 8
3.3 AES Core Transformations 8
3.3.1 SubBytes 8
3.3.2 ShiftRows 9
3.3.3 MixColumns 9
3.3.4 AddRoundKey 9
3.4 Key Expansion 9
3.5 CTR Mode Logic 10
3.6 Identified Implementation Issues 10
4. Testing 10
4.1 Discussion of Test Results 12
5. Evaluation 12
5.1 Advantages of AES-CTR 12
5.2 Disadvantages and Security Risks 12
5.3 Real-World Applications 12
5.4 Critical Evaluation of Implementation Quality 13
6. Conclusion 13
7. References 14

Table of Figures

Figure 1: AES-128 Encryption Round Structure 6

Figure 2: CTR Mode Encryption Diagram 7

Figure 3: Key Expansion (Key Schedule) Overview 9

Figure 4: CTR mode Logic..... 10

Figure 5:Test Cases..... 11

Abstract

This report documents the design, implementation, and critical evaluation of the Advanced Encryption Standard operating in Counter (CTR) mode, developed in C++.

The project aimed to produce a functional AES-128-CTR cipher capable of performing block encryption using a 128-bit key and a randomly generated nonce, and to apply the CTR mode of operation to convert the block cipher into a stream cipher suitable for arbitrary-length plaintext. The implementation encompasses the core AES transformations, including SubBytes, ShiftRows, MixColumns, and AddRoundKey, as well as the AES key expansion algorithm.

A custom AESCTR class encapsulates all functionality, exposing a clean public interface for encryption and decryption. While the encryption pathway is functional for 16-byte inputs, critical limitations were identified during testing, a fixed plaintext size constraint, and the lack of message authentication. These shortcomings are discussed in detail, and recommendations for future improvement are provided.

The report concludes that the implementation provides a sound structural and algorithmic foundation, though significant refinement is required before the system could be considered production-ready.

1. Introduction

The Advanced Encryption Standard (AES), published by the National Institute of Standards and Technology (NIST) in 2001 (FIPS PUB 197), is the most widely deployed symmetric encryption algorithm in the world, forming the cryptographic backbone of protocols such as TLS, IPSec, SSH, and numerous file-encryption utilities.

AES is a substitution-permutation network block cipher that operates on fixed 128-bit blocks of data using key sizes of 128, 192, or 256 bits. This project focuses on the 128-bit key variant, referred to as AES-128, which completes ten rounds of transformation to produce ciphertext. On its own, a block cipher can only encrypt a single fixed-size block. To handle arbitrary-length messages, a mode of operation must be employed. Counter (CTR) mode is one of the most versatile and widely recommended such modes: it generates a deterministic keystream by encrypting successive counter values and XORing the result with the plaintext, effectively converting the block cipher into a synchronous stream cipher.

1.1 Key Technical Terms

- Block Cipher: An encryption algorithm that operates on fixed-length groups (blocks) of data.
- AES-128: AES with a 128-bit key, performing 10 rounds of transformation.
- S-box (Substitution Box): A non-linear lookup table used in the SubBytes step to introduce confusion.
- Nonce: A Number Used Once; a random or unique value ensuring distinct keystreams for each encryption.
- Counter: A value appended to the nonce and incremented for each successive block.
- CTR Mode: A block cipher mode that encrypts counter values to produce a keystream for XOR with plaintext.
- Key Schedule / Key Expansion: The algorithm that derives round keys from the original cipher key.
- Round Key: A 128-bit subkey derived from the main key and used in each AES round.

1.2 Aims and Objectives

The primary aim of this project was to implement the AES-128 encryption algorithm in CTR mode using C++, without relying on third-party cryptographic libraries, in order to develop a deep understanding of the underlying mechanisms. The specific objectives were:

- To implement all four core AES round transformations (SubBytes, ShiftRows, MixColumns, AddRoundKey).
- To implement the AES-128 key expansion algorithm.
- To implement CTR mode encryption using a randomly generated nonce.
- To design a clean, encapsulated C++ class interface (AESCTR) for encryption and decryption.
- To test the implementation against valid and edge-case inputs, and critically evaluate correctness.

2. Background

2.1 The AES Algorithm

AES was selected by NIST in 2001 following an open competition to replace the aging DES standard. Internally, it operates on a 4x4 matrix of bytes called the state. For AES-128, ten rounds of four transformations are applied, with an initial key addition step. The four core operations are detailed in Table 1 below and discussed in subsequent subsections.

Table 1: AES-128 Core Round Transformations

Operation	Description & Purpose
SubBytes	Each byte in the 4x4 state matrix is substituted using the AES S-box, a non-linear bijective function derived from the multiplicative inverse in GF(2^8). This step introduces confusion by eliminating linear relationships between input and output.
ShiftRows	Rows of the state matrix are cyclically shifted leftward by their row index (row 0 unchanged, row 1 shifted left by 1, etc.). This provides inter-column diffusion, ensuring bytes from each column propagate across multiple columns in subsequent rounds.
MixColumns	Each column is treated as a polynomial over GF(2^8) and multiplied by a fixed polynomial (03x^3 + 01x^2 + 01x + 02). This operation provides diffusion across the entire column, mixing all four bytes together.
AddRoundKey	The state is XORed with the current 128-bit round key derived from the key schedule. This is the only step that incorporates the secret key and provides key-dependent transformation.

Figure 1 illustrates the structure of a single AES-128 encryption round, showing the sequential application of each transformation.

2.2 Key Expansion

AES-128 requires 11 round keys (one for the initial AddRoundKey, and one per each of the 10 rounds), each of 128 bits, yielding 176 bytes of expanded key material. The key schedule begins with the original 16-byte key and generates additional 32-bit words iteratively. Every fourth word undergoes a special transformation: RotWord cyclically rotates the four bytes, SubWord applies the S-box to each byte, and the result is XORed with a round constant (Rcon). This process ensures each round key is distinct and appears pseudorandom, preventing trivial relationships between round keys and guarding against related-key attacks.

As implemented in keyExpansion(), the process correctly initialises the first 16 bytes from the original key and iterates until 176 bytes of key material are generated. The Rcon values are pre-computed and stored in a vector, indexed by rconIndex.

2.3 CTR Mode of Operation

Counter mode (CTR) is a block cipher mode of operation standardised in NIST SP 800-38A. Rather than encrypting plaintext directly with AES, CTR mode uses AES to encrypt a sequence of counter blocks, each derived by combining a nonce with an incrementing counter value. The resulting

encrypted counter blocks form a keystream, which is XORed with the plaintext to produce ciphertext. The same operation is used for decryption, since XOR is self-inverse.

Figure 2 depicts the CTR mode encryption process. The nonce occupies the first 12 bytes of each 16-byte counter block, and the remaining 4 bytes hold the counter value, initialised to 1 and incremented in big-endian order for each subsequent block. This implementation adopts a 96-bit nonce and 32-bit counter, consistent with the NIST GCM nonce format recommendation.

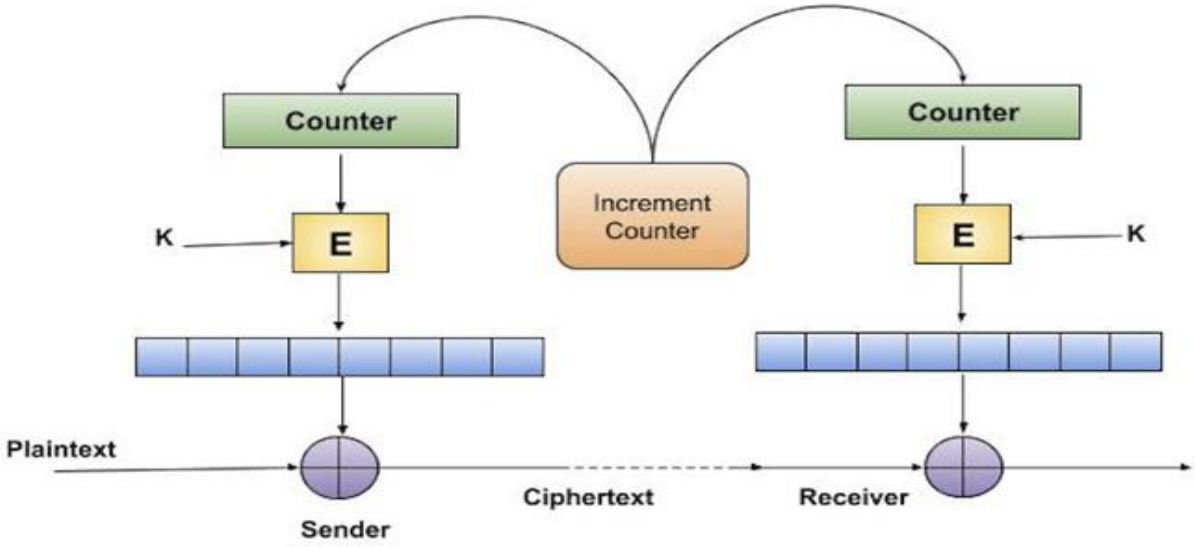
The principal advantages of CTR mode over modes such as CBC or ECB include:

(i) The ability to encrypt and decrypt blocks in parallel, since each counter value is independent.

(ii) No padding requirement, as the keystream can be truncated to match the plaintext length.

(iii) Random access to any ciphertext block without decrypting preceding blocks. These properties make CTR mode particularly suitable for high-performance applications and hardware implementations.

Figure 2 illustrates the Diagram of the Counter Mode (CTR).



Couter (CTR) Mode

3. Development

3.1 Class Architecture

The implementation encapsulates all AES-CTR functionality within a single class, AESCTR. Private member functions implement the cryptographic primitives (helper utilities, AES core transformations, key expansion, and CTR orchestration), while the public interface exposes only encrypt() and decrypt() methods. This design enforces separation of concerns and prevents misuse of internal cryptographic state. Figure 4 illustrates the high-level class architecture.

3.2 Helper Functions

A collection of utility functions underpins the higher-level cryptographic operations:

- stringToBytes() / bytesToHex() / hexToBytes(): Convert between string, hexadecimal string, and raw byte vector representations. Input to encrypt() is expected as a hex-encoded string, allowing precise specification of binary test vectors.

- generateNonce(): Employs std::random_device to generate a cryptographically seeded 12-byte nonce. While random_device is implementation-defined and may not provide cryptographic quality entropy on all platforms.

- buildNonceCounter(): Constructs the initial 16-byte counter block from the nonce. The nonce occupies bytes [0–11], and the counter field (bytes [12–15]) is initialised to 0x00000001, starting the counter at 1.

- big_endian(): Increments the 16-byte counter block in big-endian order by iterating from the most significant byte downward and propagating carry. This is the correct increment strategy for the NIST CTR construction.

- xorBlocks() / xtimes() / multiplyGF28(): Low-level XOR and Galois Field arithmetic operations. multiplyGF28() implements Russian peasant multiplication in GF(2^8), supporting MixColumns.

3.3 AES Core Transformations

The state representation uses a flat 16-element byte vector in column-major order, consistent with the AES specification. Each transformation operates on this vector:

3.3.1 SubBytes

The static sbox() method provides a constant-time (array-indexed) lookup into the AES S-box, a 256-entry table precomputed from multiplicative inverses in GF(2^8) combined with an affine

transformation. All 256 entries are hardcoded, making this implementation resistant to the performance overhead of dynamic computation.

3.3.2 ShiftRows

The shiftRows() function applies the row-shift operation using the formula result[row + 4*col] = state[row + 4*((col + row) % 4)]. This correctly implements the leftward cyclic shift of each row by its row index in column-major representation.

3.3.3 MixColumns

The mixColumns() function multiplies each column by the fixed MixColumns matrix using GF(2^8) arithmetic provided by multiplyGF28(). The fixed matrix {0x02, 0x03, 0x01, 0x01, ...} correctly implements the standard AES MixColumns polynomial multiplication.

3.3.4 AddRoundKey

The addRoundKey() function performs a byte-wise XOR between the current state and the 16-byte round key, as required by the AES specification. getRoundKey() extracts the appropriate 16-byte subkey from the expanded key array by offset.

3.4 Key Expansion

The keyExpansion() function implements the AES-128 key schedule, generating 176 bytes of round key material. The implementation correctly: initializes the first 16 bytes from the cipher key; applies RotWord, SubWord, and Rcon XOR to every fourth word and accumulates key material by XORing with the word 16 bytes prior. All ten required Rcon values are precomputed in a vector, indexed sequentially.

```
vector<uint8_t> keyExpansion(const vector<uint8_t> &key)
{
    const int Nk = 4;
    const int Nb = 4;
    const int Nr = 10;
    const int wordSize = 4;

    if (key.size() != 16)
        throw invalid_argument("Key must be 16 bytes for AES-128");

    vector<uint8_t> expanded_key((Nb * (Nr + 1)) * wordSize);
    vector<uint8_t> Rcon = {
        0x01, 0x02, 0x04, 0x08, 0x10,
        0x20, 0x40, 0x80, 0x1B, 0x36};
    vector<uint8_t> temp(4);

    for (size_t i = 0; i < 16; i++)
        expanded_key[i] = key[i];

    int generated_bytes = 16, rconIndex = 0;

    while (generated_bytes < expanded_key.size())
    {
        for (size_t i = 0; i < 4; i++)
            temp[i] = expanded_key[generated_bytes - 4 + i];

        if (generated_bytes % (Nk * 4) == 0)
        {
            temp = subWord(rotWord(temp));
            temp[0] ^= Rcon[rconIndex++];
        }

        for (size_t i = 0; i < 4; i++)
        {
            expanded_key[generated_bytes] = expanded_key[generated_bytes - 16] ^ temp[i];
            generated_bytes++;
        }
    }

    return expanded_key;
}
```

3.5 CTR Mode Logic

The private AES_CTR_private() method implements CTR mode encryption. For each 16-byte block (or partial block) of input, the current counter value (nonceCounter) is encrypted with AES_encrypt_block() to produce a 16-byte keystream block. The keystream is XORed byte-by-byte with the corresponding plaintext bytes, and the counter is then incremented with big_endian():

```
vector<uint8_t> AES_CTR_private(const vector<uint8_t> &input, const vector<uint8_t> &key, vector<uint8_t> nonceCounter)
{
    vector<uint8_t> cipher_encipher;

    for (size_t i = 0; i < input.size(); i += 16)
    {
        size_t blockSize = min((size_t)16, input.size() - i);

        vector<uint8_t> keyStream = AES_encrypt_block(nonceCounter, key);

        for (size_t j = 0; j < blockSize; j++)
            cipher_encipher.push_back(input[i + j] ^ keyStream[j]);

        big_endian(nonceCounter);
    }

    return cipher_encipher;
}
```

The public encrypt() method validates the plaintext and key, generates a random nonce, builds the initial counter block, invokes AES_CTR_private(), and returns the result as a hex string prefixed with the nonce, separated by a colon (nonce_hex : ciphertext_hex).

3.6 Identified Implementation Issues

Issue 3 – Absence of Message Authentication: The implementation provides no integrity protection. An adversary can flip bits in the ciphertext, causing predictable and undetected changes to the recovered plaintext. This is a known vulnerability of unauthenticated CTR mode, which is typically mitigated by using authenticated encryption modes such as AES-GCM or appending an HMAC.

Issue 4 – random_device Entropy Quality: std::random_device is not guaranteed to be cryptographically secure across all C++ compilers and platforms. On some Windows implementations it may be deterministic. A production implementation should use OS-level cryptographic random sources (e.g., /dev/urandom on Linux, CryptGenRandom on Windows, or the BCrypt API).

4. Testing

A structured suite of tests was designed to evaluate the correctness and robustness of the implementation across normal, boundary, and adversarial input conditions. Table 2 the key test cases, their expected outcomes, and whether the implementation passed or failed each case.

Table 2: Test Cases

```
--- Encryption Mode ---
[1] Input HEX
[2] Input TEXT
Choose: 1
Enter 16-byte key (hex, 32 chars): 0f1571c947d9e8590cb7add6af7f6798
Enter plaintext (hex): ffffffffffffffffffffffffffffffffffffffffffffffff

========== RESULT ==========
Nonce Key(hex): c71bdb40964236c11882af89 :  Cipher Text(hex): 91c074396003d7e9486df2d8e6434a0c17b61ec7f45d28ad
Time: 0.2509 ms
============================
Copy that to decrypt: c71bdb40964236c11882af89:91c074396003d7e9486df2d8e6434a0c17b61ec7f45d28ad

=====================================
              MAIN MENU
=====================================
[1] Encrypt
[2] Decrypt
[3] Exit
-------------------------------------
Choose: 2

--- Decryption Mode ---
[1] Output HEX
[2] Output TEXT
Choose: 1
Enter 16-byte key (hex, 32 chars): 0f1571c947d9e8590cb7add6af7f6798
Enter (nonce:ciphertext): c71bdb40964236c11882af89:91c074396003d7e9486df2d8e6434a0c17b61ec7f45d28ad

========== RESULT ==========
Plain Text(hex): ffffffffffffffffffffffffffffffffffffffffffffffff
Time: 0.1021 ms
============================
```

```
--- Encryption Mode ---
[1] Input HEX
[2] Input TEXT
Choose: 1
Enter 16-byte key (hex, 32 chars): 00112233445566778899aabbccddeeff
Enter plaintext (hex): 00112233445566778899aabbccddeeff

========== RESULT ==========
Nonce Key(hex): 5f8fa84c04b32401a76ac817 :  Cipher Text(hex): 7e83e8fdde9202903c090f36f3c68830
Time: 0.205 ms
============================
Copy that to decrypt: 5f8fa84c04b32401a76ac817:7e83e8fdde9202903c090f36f3c68830

=====================================
              MAIN MENU
=====================================
[1] Encrypt
[2] Decrypt
[3] Exit
-------------------------------------
Choose: 2

--- Decryption Mode ---
[1] Output HEX
[2] Output TEXT
Choose: 1
Enter 16-byte key (hex, 32 chars): 00112233445566778899aabbccddeeff
Enter (nonce:ciphertext): 5f8fa84c04b32401a76ac817:7e83e8fdde9202903c090f36f3c68830

========== RESULT ==========
Plain Text(hex): 00112233445566778899aabbccddeeff
Time: 0.0842 ms
```

5. Evaluation

5.1 Advantages of AES-CTR

AES in Counter mode offers several compelling advantages that justify its adoption in modern cryptographic protocols:

- Parallelism: Because each counter block is encrypted independently, CTR mode is trivially parallel for both encryption and decryption. This makes it highly suitable for multi-core processors and GPU implementations, delivering significant performance gains over sequential modes such as CBC.
- No Padding Required: CTR mode generates a keystream that is XORed with the plaintext; the keystream is simply truncated to the plaintext length. This eliminates the padding overhead and associated vulnerabilities.
- Random Access: Any block of the ciphertext can be decrypted independently by computing only the keystream for that block's counter value, enabling efficient decryption of large files with seek support.
- Pre-computation: The keystream can be generated in advance, independently of the plaintext, allowing latency-sensitive applications to pre-compute and store keystream material.

5.2 Disadvantages and Security Risks

- No Authentication: CTR mode provides confidentiality only; it offers no integrity or authenticity guarantee. A malicious actor intercepting the ciphertext may perform bit-flipping attacks with predictable effects on the decrypted plaintext without detection. This is the most significant practical weakness of the mode in isolation.

- Nonce Reuse Catastrophe: Reusing the same nonce with the same key produces an identical keystream, allowing an attacker who observes two ciphertexts to compute their XOR, which equals the XOR of the two plaintexts. This can completely compromise confidentiality. The implementation mitigates this through random nonce generation, though the entropy source is not cryptographically certified.

- Key Stream Exposure: If a keystream block is ever exposed due to a plaintext-known attack, it trivially reveals the corresponding ciphertext block's plaintext for any future message using the same counter and key.

5.3 Real-World Applications

CTR mode forms the foundation of AES-GCM (Galois/Counter Mode), the dominant authenticated encryption scheme in TLS 1.3, IPSec, and SSH. It is also used in disk encryption variants and in certain stream cipher constructions in IoT devices where hardware AES acceleration is available.

6. Conclusion

This project successfully implemented the core AES-128 encryption algorithm and Counter mode of operation in C++, encapsulated within the AESCTR class.

The SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations were implemented according to the FIPS 197 specification, and the key expansion algorithm correctly generates the 176 bytes of round key material required for AES-128. The CTR mode keystream generation and XOR application were demonstrated to be functional for the supported 16-byte input size.

Deep practical understanding of finite field arithmetic in GF(2^8), the structural design of substitution-permutation network ciphers, the operation of counter-based modes, and the critical role of nonce management in symmetric encryption. The project also highlighted the distinction between a cryptographically correct algorithm and a cryptographically secure implementation, particularly with regard to authentication, side-channel resistance, and entropy quality.

Future improvements to the implementation should priorities the following:

- Integrate a message authentication code, ideally by extending the implementation to AES-GCM, which combines CTR mode with GHASH-based authentication.

- Replace std::random_device with a certified OS cryptographic random source to guarantee nonce unpredictability.

7. References

Advanced Encryption Standard (AES) - GeeksforGeeks

Advanced Encryption Standard (AES) FIPS 197 Federal Information Processing Standards Publication

Cryptography - Counter (CTR) Mode